

THE COMPLETE REFERENCE

DocuMind

*The Complete Reference · production-grade multimodal RAG
over PDFs, end to end*

Version 0.1

DocuMind contributors · June 2026

Colophon

DocuMind · The Complete Reference describes the architecture, implementation, and operations of the open-source DocuMind project (<https://github.com/dyh1265/DocuMind>). This book is generated directly from the repository by the script `scripts/generate_book.py` and is byte-reproducible: re-running the script on the same source tree produces an identical PDF. The wall-clock fields the PDF format demands are pinned via `SOURCE_DATE_EPOCH`.

Code excerpts in this book reference the master branch at the time of generation. Cross-references to `backend.*` and `frontend/` paths map directly to the repository layout described in Appendix A.

The book is released under the MIT license, the same terms as the source repository. You are free to copy, fork, adapt, and ship it.

Type and layout

Body text is set in Helvetica at 10.5pt with 14pt leading on an A4 page. Code is set in Courier at 9pt. Headings are Helvetica Bold. Layout is produced by PyMuPDF (fitz) using the built-in PDF type-1 fonts; no external font files are required.

Preface

Most RAG repositories are notebooks. They show that the idea works and stop there. The thing they consistently miss is what makes a real product: the second day of operation.

On the first day you can stitch together a vector store, a chunker, an embedding model, and a chat completion call, and get an answer. On the second day the questions arrive: How do you tell whether the answer is right? How do you keep it right when you change the chunker? What happens when the PDF has a 14-column table? What does the user see when the LLM is unreachable? Where do PII redactions happen, and at which stage? What gets cached, and what doesn't? How do you make ingest of a large folder of PDFs not block the chat UI? What does the latency budget look like end-to-end, and which knob buys you the most quality per millisecond?

DocuMind is one answer to that second day. It is not the best possible answer; it is an opinionated, working one. This book exists so that, by the time you have read it, those questions and their trade-offs are visible to you in concrete code paths rather than vague principles.

Who this book is for

- Engineers building a RAG system over structured documents who want to see how the pieces fit before deciding which to keep and which to swap.
- Reviewers evaluating the repository for portfolio purposes who want a single document explaining what is here and why.
- Operators running DocuMind in production who need a reference for every configuration knob, metric, and failure mode.
- Contributors who want to understand the design intent of a module before changing it.

How to read it

The book is structured as a single read-through. Chapters 1 and 2 give the motivation and the system topology · read them in order. Chapters 3 through 6 follow a document and a question through the pipeline, layer by layer; you can read them in order or jump to the layer you are debugging. Chapters 7 through 10 cover the operational surface · evaluation, observability, scaling, deployment. Chapters 11 and 12, and the appendices, are reference material: dip into them when you need an HTTP endpoint or a config flag. Chapter 13 covers an alternative ingest source · YouTube lectures · that rides the same pipeline as PDFs.

CONVENTION

Throughout, file paths are written with forward slashes and rooted at the repository, e.g. `backend/core/pipeline.py`. Environment variable names are in `SCREAMING_SNAKE_CASE`; Python settings on the `Settings` class are in `snake_case`.

Table of Contents

1. What DocuMind Is and Why	7
The problem	7
The shape of the solution	7
When this design is a good fit	8
When something else might be a better fit	8
A two-minute tour	8
2. The Big Picture	10
Service topology	10
Components and their responsibilities	10
Request flow: a single question	11
Request flow: bulk PDF ingest	11
Why these boundaries	12
Configuration surface	12
3. The Ingestion Pipeline	13
Parsing	13
Enrichment	14
Embedding	14
Storage	14
Progress events	15
4. Retrieval Strategy	16
The high-level flow	16
Modality routing	16
Hybrid retrieval (text only)	17
Multimodal fusion across collections	17
Parent-chunk expansion	17
Labeled-asset retrieval	17
Optional rerankers	18
ColPali (page-image retrieval)	18
Chunk filters	18
Why this stack	18
5. Answer Generation	19
The cited-context prompt	19
Citation objects	19
LLM providers	20
Fallback paths	20
Streaming	20
6. Guardrails	21
PII redaction	21

Taxonomy conformity	21
Rate limiting	22
CORS	22
7. Evaluation	23
The golden set	23
Metrics	23
CI gates	24
Workflows	24
Reproducing the benchmarks locally	24
8. Observability	26
Traces · OpenTelemetry to Jaeger	26
Metrics · Prometheus + Grafana	26
Structured logs	26
Health and readiness	27
9. Scaling Out	28
Why a worker tier	28
The bulk-ingest protocol	28
Deduplication	28
Caching	29
Sizing notes	29
10. Deployment	30
Local Docker Compose	30
GPU build	30
Fly.io	30
nginx in front	31
CI / CD	31
11. API Reference	32
Health and metrics	32
Query	32
Ingest	32
Admin	33
12. Configuration Reference	34
LLM provider	34
Embeddings and retrieval models	34
Retrieval toggles	34
Vector DB and queue	35
OCR and parsing	35
YouTube lecture ingest	35
API, guardrails, observability	35
Evaluation thresholds	36
Reproducible builds	36
13. Ingesting YouTube Lectures	37
Two modalities from one URL	37

The ingest path	37
Slide-scoped questions	38
Citations and preview	38
Operating it	38
Limitations	39
Appendix A · Repository Layout	40
Appendix B · Troubleshooting	42
/query returns 503 with a torch upgrade message	42
/query returns 'Internal Server Error'	42
PDFParseError on a specific page	42
Long PDF upload returns 504 with raw HTML in the UI	42
Bulk ingest hangs on a single file	42
Sample PDF differs across machines	43
Appendix C · Glossary	44

CHAPTER 1

What DocuMind Is and Why

The problem

Retrieval-Augmented Generation is the practical answer to the hardest single failure mode of large language models: they confidently produce statements they cannot back up. The fix is mechanical · find passages from a trusted corpus that look relevant to the question, show them to the model, and require the model to answer only with what is in those passages, citing them.

Once you take that idea seriously the implementation expands quickly. Where do the passages come from? PDFs from your finance team, slide decks from product marketing, scientific papers with figures and tables, scanned forms with no embedded text at all. How do you split them so the retriever can find the right span without losing context? How do you handle a question whose answer is in a chart caption, not the prose? How do you keep retrieval honest after a year of drift in the underlying corpus? What does the user see when the LLM is down, when the embedding model OOMs on a large doc, when the answer violates an internal taxonomy?

DocuMind is a working system that answers all of those questions in code. It is deliberately not a black-box library you import and configure; it is a small, opinionated stack of services you can read, swap, and operate.

The shape of the solution

DocuMind ingests PDFs as four kinds of content · text passages, tables, figures, and, optionally, whole page images · and stores each kind in its own vector collection. At query time it retrieves across all of them in parallel, fuses the results with rank-based reciprocal rank fusion, optionally reranks with a cross-encoder, and asks an LLM (OpenAI or a local Ollama) to answer with inline citations that point back to the chunk, page number, and bounding box. The frontend renders those citations as clickable badges that open the source page.

The stack as it ships:

- Frontend: React + Vite with a chat UI, document preview, and bulk-upload progress.
- API: FastAPI with rate limiting, PII redaction, taxonomy guardrails, OpenTelemetry tracing, and a Server-Sent Events progress stream for long ingests.
- Pipeline: a single RAGPipeline class that the API, eval suite, and Celery worker all share.
- Ingestion: PyMuPDF + pdfplumber + Tesseract OCR for parsing; BGE-M3 + CLIP +

(optional) ColPali for embeddings.

- Vector store: Qdrant with four collections · text_chunks, table_chunks, figure_chunks, page_chunks.
- Queue and cache: Redis as both the Celery broker and the embedding cache (chunk fingerprint keyed, 7-day TTL).
- Workers: Celery for bulk ingest, with LSH dedup against the Qdrant content hash.
- Observability: Prometheus, Grafana, Jaeger; an OpenTelemetry collector wires traces through ingest and retrieval.
- Evaluation: a hand-curated golden set with CI-gated thresholds for recall@5, hit@5, MRR, p95 retrieval latency, and answer keyword coverage.

When this design is a good fit

- Your corpus is structured documents · PDFs with figures, tables, equations, layouts · not just plain prose.
- You want citations the user can click through to the source page, not just an answer.
- You can tolerate a multi-service stack (Qdrant + Redis + an API and worker) in exchange for proper observability, dedup, and bulk-ingest semantics.
- You care about evaluation as a first-class concern, not an afterthought.
- You expect to swap individual components (reranker, embedder, LLM) without rewriting the pipeline.

When something else might be a better fit

- You have plain-text knowledge-base articles · a single-collection vector index and a small library are probably overkill in reverse here.
- You need sub-100 millisecond p95 · the multimodal fusion plus optional reranker stage is honest about its latency budget.
- You need streaming token-by-token UX · currently the generator returns the full answer (extending to streaming is straightforward and noted in Chapter 5).
- You are working in a heavily regulated context that forbids any LLM call · DocuMind can run retrieve-only, but its main loop assumes a generation step.

A two-minute tour

The fastest way to internalize the project is to run it. From a clean checkout on a host with Docker:

```
cp .env.example .env
# put your OPENAI_API_KEY in .env
cd docker
docker compose --profile production up -d --build
```

Open <http://localhost:8080>, drag any PDF onto the upload card, watch the ingest progress stages (uploading, parsing, chunking, embedding, indexing), then ask a question. The answer comes back with citation badges; clicking one opens the source panel and jumps the embedded PDF viewer to the right page.

GPU VS CPU

The default Compose stack runs on CPU. To use an NVIDIA GPU, add the GPU overlay:
`docker compose -f docker-compose.yml -f docker-compose.gpu.yml --profile production up -d --build`. This picks the cu124 PyTorch wheels and sets `EMBEDDING_DEVICE=cuda`. See Chapter 10 for the full deployment guide.

CHAPTER 2

The Big Picture

Service topology

DocuMind is split into a stateless API layer, a stateful retrieval layer, a worker tier for bulk ingest, and an observability sidecar. Each piece is a separate container so they can be scaled or replaced independently. The diagram below sketches the flow; the next subsections name every box.

```

Browser --> React/Vite --> nginx --> FastAPI (rag-api)
                                   |
                                   v
                               RAGPipeline
                               +-- Ingestion --> Qdrant
                               +-- Retrieval  --> Qdrant
                               +-- Taxonomy guard
                               +-- LLM (OpenAI / Ollama)

FastAPI ---> Celery worker (Redis broker) --> Qdrant
FastAPI ---> OpenTelemetry collector --> Jaeger
FastAPI ---> /metrics <-- Prometheus --> Grafana

```

Components and their responsibilities

Frontend	frontend/ · chat UI, citation rendering, document preview, bulk-upload progress. Vite dev or nginx in production.
API	backend/api/ · FastAPI routers (query, ingest, bulk_ingest, admin, health), rate limiting (slowapi), PII guardrails, CORS, OpenTelemetry.
Pipeline	backend/core/pipeline.py · single RAGPipeline class. The API, eval harness, and Celery worker all run through it instead of wiring components by hand.
Ingestion	backend/ingestion/ · PDF text/table/figure parsers, Tesseract OCR, BGE-M3 + CLIP + ColPali embedders, the Qdrant store wrapper.
Retrieval	backend/retrieval/ · hybrid BM25 + dense, multimodal RRF fusion, parent-chunk expansion, cross-encoder + FlashRank rerankers, chunk filters.
Generation	backend/generation/answer_generator.py · OpenAI + Ollama backends, citation building, fallback when the LLM is unreachable.

Scaling	backend/scaling/ · Celery bulk-ingest tasks, Redis embedding cache, fingerprint-based dedup.
Taxonomy	backend/taxonomy/ · RDF taxonomy validation, conformity checking, fuzzy entity linking. Can warn or hard-block forbidden answers.
Vector DB	Qdrant · four collections: text_chunks, table_chunks, figure_chunks, page_chunks (created only when ColPali is enabled).
Queue + cache	Redis · Celery broker for bulk jobs, embed cache keyed by chunk fingerprint.
Observability	Prometheus, Grafana, Jaeger · /metrics scrape, dashboards, OTLP traces for ingest and retrieval.

Request flow: a single question

1. Browser issues POST /api/query with the doc_id, query, and provider; nginx proxies to rag-api on the internal network.
2. FastAPI rate-limits the request per IP via slowapi, then validates the QueryBody schema.
3. Optional PII redaction (Presidio + spaCy) runs on the query text. The PIIRedactor is constructed with an explicit en_core_web_sm NLP engine pinned to English · see Chapter 6 for why.
4. RAGPipeline.query runs the full multimodal retrieval (text, tables, figures, optional page images) and returns ranked contexts.
5. If retrieve_only is false, AnswerGenerator builds the cited prompt and calls OpenAI or Ollama. The system prompt is strict: answer ONLY from the numbered context passages, cite sources inline as [1], [2].
6. The taxonomy guard inspects the answer; if TAXONOMY_BLOCK_FORBIDDEN=true and a forbidden classification appears, it replaces the answer with a structured refusal.
7. The QueryResponse · answer + citations + retrieved contexts + latency · is returned. OTLP spans for retrieve and generate land in Jaeger; Prometheus counters for /query are incremented.

Request flow: bulk PDF ingest

1. Browser issues POST /api/ingest/bulk/start with the folder name and total file count; the API creates a job record in Redis and returns a job_id.
2. Each PDF is uploaded with POST /api/ingest/bulk/{job_id}/files; the API saves to data/raw/uploads/bulk/{job_id}/ on a shared volume.
3. POST /api/ingest/bulk/{job_id}/run enqueues one Celery task per uploaded PDF.
4. Workers (backend.scaling.workers) pull tasks, fingerprint each PDF, dedup against an in-process LSH cache plus the Qdrant content hash, and call RAGPipeline.ingest on new docs.
5. Workers update Redis (bulk:job:{id}:status) and emit Prometheus metrics on INGEST_METRICS_PORT.
6. Browser polls GET /api/ingest/bulk/{job_id} every second and renders the progress bar; on completion the document is selectable from the sidebar.

Why these boundaries

- API to pipeline: keeping the FastAPI layer thin means the same RAGPipeline runs in the eval harness, the Celery worker, and the API without any duplication. Tests do not have to spin up FastAPI.
- Worker to API: bulk ingest can saturate a CPU for minutes. Putting it on Celery + Redis means a single PDF upload from the chat UI cannot get queued behind a 500-PDF batch.
- Four Qdrant collections: text, tables, figures, and (optional) page images are embedded by different models and retrieved with different query hints. One collection per modality keeps the search index dense and the fusion logic in MultiModalRetriever explicit.
- OpenTelemetry: the same trace ID propagates from the FastAPI request through the embedder and back, which makes 'why was that query slow?' answerable from Jaeger without bisecting the code.

Configuration surface

Every backend toggle lives in `backend/core/config.py` as a single pydantic-settings `Settings` class. There are no scattered `os.getenv()` calls. The full table is Chapter 12; the rule of thumb is that any knob worth flipping at runtime appears there, with an alias matching the `SCREAMING_SNAKE_CASE` environment variable.

SINGLE SOURCE OF TRUTH

If you find yourself reading a value from `os.environ` in a backend module, that is a bug · promote it to `Settings` instead. Both the eval harness and the API rely on `get_settings()` returning the same instance per process (it is wrapped in `lru_cache`).

CHAPTER 3

The Ingestion Pipeline

Ingestion is the four-step path from a PDF on disk to vectors in Qdrant: parse, enrich, embed, store. Each step is in its own module and is fully overridable. The whole thing lives behind `RAGPipeline.ingest(path)` and emits structured progress events when called via `POST /ingest/stream` · see Chapter 11 for the SSE protocol.

NOT ONLY PDFS

PDFs are the primary source, but the same pipeline also ingests YouTube lectures: a video's transcript and slides are reduced to the same chunks and collections described here. That path is covered in Chapter 13.

Parsing

`backend.ingestion.parsers` registers one parser per modality. The `ParserRegistry` dispatches by file extension and chunk type. For PDFs the registry runs four parsers in sequence and merges their outputs.

PDFTextParser	PyMuPDF based. Extracts running text per page; inherits section headings to populate <code>section_path</code> on each chunk.
TableParser	pdfplumber. Detects ruled tables via line-intersection; falls back to text extraction when no ruling is present.
FigureParser	Caption-aware bounding boxes around image regions; labels (e.g. 'Figure 3: ...') are captured for labeled-asset retrieval.
PageImageParser	Whole-page renders, only used when <code>USE_COLPALI=true</code> . Lets ColPali embed layout directly.

Document IDs are deterministic

`stable_doc_id()` in `backend/ingestion/parsers/base_parser.py` hashes the path from the raw/ anchor down, after normalizing separators. The same PDF re-ingested from a Windows host (`C:\Users\...\raw\file.pdf`), a Linux container (`/app/data/raw/file.pdf`), or a relative path (`data/raw/file.pdf`) all upsert to the same Qdrant row instead of duplicating. This is tested cross-platform in CI.

OCR for image-only pages

When `USE_OCR=true` (default), Tesseract via `pytesseract` runs against pages that produced no

parseable text and no images either. The fallback path is explicit: `PDFParseError` is raised when a page is neither parseable nor renderable, with a hint that the document may be scanned-only without OCR. The user-visible error surface in the frontend reflects this · empty pages don't silently disappear.

Enrichment

`backend.retrieval.preprocessing` runs at ingest time so the cost is paid once per document, not per query. Each step is a feature flag:

USE_RECURSIVE_CHUNKER	Token-aware splitter (max 512, overlap 64). Off by default; recommended on for production.
USE_SEMANTIC_CHUNKER	Alternative to recursive · splits on cosine-distance boundaries between consecutive sentences. Threshold tunable via <code>SEMANTIC_CHUNK_THRESHOLD</code> (default 0.75).
USE_SECTION_PATHS	Attach the heading chain (Executive Summary > Q4 Highlights) to each chunk.
USE_CONTEXT_ENRICHMENT	Pre-compute the [context] previous sentence + chunk + next sentence block used to disambiguate retrieved fragments.
USE_PII_REDACTION_ON_INGEST	Presidio + spaCy redact emails, phone numbers, credit cards, IBANs, IPs, SSNs, passports, and a few more, before the vector store ever sees them.

Embedding

`backend.ingestion.embeddings` routes chunks to one of three embedders based on chunk type. The text and image branches are required; the ColPali branch is opt-in and useful for layout-heavy documents.

TextEmbedder	BAAI/bge-m3 by default (1024-d). Multilingual, strong on long-context retrieval. Runs on cuda when <code>EMBEDDING_DEVICE=cuda</code> .
ImageEmbedder	laion/CLIP-ViT-B-32-laion2B-s34B-b79K. Embeds figures and table images into a shared text/image space so figure retrieval works with text queries.
ColPaliEmbedder	vidore/colqwen2-v1.0 by default (vidore/colSmol-500M for CPU). Embeds whole rendered pages; used when <code>USE_COLPALI=true</code> .

`embed_chunks` batches by modality and caches by chunk fingerprint in Redis (`embedding_cache_ttl_seconds` defaults to 7 days). A re-ingest of an unchanged PDF is essentially free; a re-ingest after a chunker change pays only for the new chunks.

TORCH AND TRANSFORMERS COMPATIBILITY

transformers 5.x refuses to load model weights under torch < 2.6 (CVE-2025-32434). The Docker image pins torch ≥ 2.6 and defaults to the cu124 wheel for GPU builds; the cu121 index only ships torch 2.5.x. Mismatched stacks surface as 503 on /query with an actionable message instead of a generic Internal Server Error.

Storage

QdrantStore in backend/ingestion/stores/qdrant_store.py writes one row per chunk into the matching collection (text_chunks, table_chunks, figure_chunks, page_chunks). Each row carries enough metadata for the frontend's 'open the citation' action to deep-link into the PDF viewer:

- doc_id · the deterministic stable_doc_id, used as the partition key for all retrieval.
- source_path · original PDF path (relative to data/raw/).
- page_number · 1-indexed page on which the chunk appears.
- chunk_type · text / table / figure / page_image.
- section_path · the heading chain when USE_SECTION_PATHS=true.
- bbox · bounding box for figures and tables, used by the frontend overlay.
- content · the text or, for images, a textual surrogate (caption + alt text).
- content_hash · SHA-256 of the raw chunk text, used for cross-job dedup at bulk-ingest time.

Progress events

RAGPipeline.ingest accepts an optional on_progress callback that the POST /ingest/stream endpoint wires to a Server-Sent Events stream. Stage names are stable: parsing, enriching, redacting, embedding, indexing. The detail payload includes chunk_count, vector_count, and vectors_by_collection where applicable. The frontend uses these events to drive the upload spinner and a real per-stage label.

```
event: progress
data: {"stage":"parsing","message":"Reading PDF pages..."}

event: progress
data: {"stage":"embedding","message":"Embedding 8733 chunks...","detail":{"chunk_count":8733}}

event: done
data: {"doc_id":"5c77f0e3a96a4de4","chunk_count":8733, ... }
```

CHAPTER 4

Retrieval Strategy

Retrieval is the bulk of DocuMind's complexity. The goal: given a natural-language question, return the smallest set of chunks (text, tables, figures, optionally page images) that lets the LLM answer with citations. This chapter walks every stage that lives under backend/retrieval/.

The high-level flow

```

question
|
+-- modality hint? (Figure 3, Table 1, page N)
v
per-collection search (text, tables, figures, pages -- parallel)
|
v
hybrid: BM25 + dense within text_chunks
|
v
Reciprocal Rank Fusion across collections
|
v
parent-chunk expansion
|
v
(optional) cross-encoder or FlashRank rerank
|
v
top-K -> AnswerGenerator

```

Modality routing

MultiModalRetriever uses cheap regex/keyword heuristics to detect modality hints in the question. The hint biases · it does not gate · so a question like 'what does Figure 3 show' still searches text in case the answer is in the figure's caption rather than the figure itself.

```

'table', 'row',          boost table_chunks
'column', 'Table N'

```

<code>'figure', 'chart', 'plot', 'Figure N'</code>	boost figure_chunks
<code>'on page N', 'appendix N'</code>	apply page_number Qdrant filter
<code>no hint</code>	all collections searched equally

Hybrid retrieval (text only)

When `USE_HYBRID=true` (off by default; recommended on for production), `HybridRetriever` combines dense and sparse search:

- Dense: top-K nearest neighbours from Qdrant on the BGE-M3 embedding of the question.
- Sparse: BM25 over the same chunks, using rank-bm25 with a tokenizer that handles numeric tokens and hyphenated terms.

The two lists are merged with Reciprocal Rank Fusion:

```
score(c) = sum over retrievers r of 1 / (k + rank_r(c))      where k = 60
```

RRF is rank-based rather than score-based, so we do not need to normalize BM25 and cosine scales · a long-standing source of fragility in hybrid search. Why both? BGE-M3 dense vectors handle synonyms ('revenue' is close to 'income') and paraphrasing; BM25 handles exact strings the embedder does not care about ('Q4 2024', '\$191.7M'). The combination beats either alone on the golden set.

Multimodal fusion across collections

After per-collection retrieval, RRF runs again · this time fusing the top results from `text_chunks`, `table_chunks`, `figure_chunks`, and (if enabled) `page_chunks`. Modality hints simply add a small constant boost to the relevant collection's rank. The output is a single ranked list of `RetrievedContext` objects.

Parent-chunk expansion

When `USE_PARENT_EXPAND=true` (default), a hit on a small 'child' chunk pulls in its neighbouring siblings to form a richer context. This solves the classic chunking trade-off:

- Small chunks (around 256 tokens) rank well because they are focused, but they truncate the surrounding evidence.
- Large chunks (around 1024 tokens) carry more context but rank worse because the signal-to-noise drops.

DocuMind embeds the small chunks but, on a hit, expands to the parent passage. The expansion logic lives in `backend/retrieval/parent_expand.py`.

Labeled-asset retrieval

Tables and figures often have explicit labels · Table 1, Figure 3. `backend/retrieval/asset_refs.py`

detects these references in both the query and the chunk metadata, and adds them as a high-precedence path on top of the vector/BM25/RRF stack. This means 'What does Figure 3 show?' directly retrieves the chunk that was labeled 'Figure 3' by the parser at ingest time · no embedding match required.

Optional rerankers

Cross-encoder	BAAI/bge-reranker-v2-m3, ~250 ms for 10 chunks on CPU. Best quality, ~10× slower than no reranker.
FlashRank	ms-marco-MiniLM-L-12-v2, ~25 ms for 10 chunks. Solid quality, almost free latency. Set USE_FLASHRANK=true.

Both rerankers shrink the retrieved-context list from default_top_k (5) down to reranker_top_n (3) before the LLM call, which also makes the LLM cheaper.

ColPali (page-image retrieval)

When USE_COLPALI=true, DocuMind also embeds whole rendered pages with a vision-language model (default vidore/colqwen2-v1.0). This is useful for documents where the layout itself encodes information · forms, multi-column scientific papers, slide decks · where the text embedder loses information that ColPali keeps. Practical notes: ColPali needs a GPU to be tolerable (use vidore/colSmol-500M for CPU), and it is purely additive · the page_chunks collection participates in RRF fusion just like text, tables, and figures.

Chunk filters

backend/retrieval/chunk_filters.py runs cheap post-retrieval filters before the LLM call:

- `is_substantive_content`: drops chunks whose visible text is whitespace, single characters, or page numbers · common artefacts from OCR on scanned PDFs.
- Dedup by `chunk_id`: same chunk surfacing from both dense and sparse retrieval is collapsed.

If every retrieved chunk fails `is_substantive_content`, the generator short-circuits to the 'no searchable text' fallback rather than asking the LLM to answer from nothing.

Why this stack

Hybrid (BM25 + dense) + RRF	Real questions cite specific numbers and identifiers. Dense alone misses them; RRF without rescaling is robust.
Separate collection per modality	Different embedding models per modality means different vector dimensionalities. Qdrant needs separate collections.
Parent expansion	Decouples what we index for ranking from what we feed the LLM.
Optional rerankers	Quality / latency knob the operator owns. CI runs without; production can flip the toggle.
ColPali behind a flag	Costs a GPU; only worth it for layout-heavy corpora.

CHAPTER 5

Answer Generation

The cited-context prompt

AnswerGenerator builds a numbered context block where every entry names its page and chunk type. The LLM is given a strict system prompt: answer ONLY using the numbered context passages, cite sources inline as [1], [2]. If the context is insufficient, the model is instructed to say so rather than guess.

[1] (page 2, text)
Executive Summary. Acme Corp delivered strong financial performance in 2024. Total revenue reached \$191.7M, a 24% year-over-year increase.

[2] (page 3, table)
Quarter | Revenue (\$M) | YoY Growth | Net Margin
Q1 2024 | 41.2 | +18% | 9.4%
Q2 2024 | 46.7 | +21% | 10.8%
Q3 2024 | 49.2 | +25% | 11.5%
Q4 2024 | 54.6 | +32% | 12.7%

Citation objects

The response is wrapped in a QueryResponse with Citation objects that carry doc_id, source_path, page_number, chunk_id, and a trimmed excerpt. The frontend uses those to render clickable citation badges and open the source panel.

```
{
  "answer": "Q4 2024 revenue was $54.6M [2].",
  "citations": [
    {
      "doc_id": "ed7d53f9b08caa39",
      "source_path": "data/raw/sample_report.pdf",
      "page_number": 3,
      "chunk_id": "table_3_0",
      "excerpt": "Q4 2024 | 54.6 | +32% | 12.7%"
    }
  ],
  "latency_ms": 1842.3
}
```

```
}
```

LLM providers

<code>openai</code>	OPENAI_API_KEY required; OPENAI_MODEL defaults to gpt-4o. The Vision model (gpt-4o by default) is reused for figure-bearing answers.
<code>ollama</code>	OLLAMA_BASE_URL points to a local Ollama instance. In Docker, set to <code>http://host.docker.internal:11434</code> . The default model is llama3.2; override via the request body.

The provider is selected per request via the provider field on the query body. The frontend exposes it as a dropdown in the sidebar so you can switch between OpenAI and a local Ollama without restarting.

Fallback paths

- No retrieved context: the generator returns a friendly fallback ('This document may be image-only; re-upload to run OCR') instead of hallucinating from nothing.
- Only non-substantive context: same fallback fires when every retrieved chunk fails `is_substantive_content`.
- LLM unreachable: the generator catches the connection error and returns the retrieved contexts with `answer='LLM provider unavailable; see citations below'` so the user still gets sources.
- Taxonomy block: with `TAXONOMY_BLOCK_FORBIDDEN=true`, an answer that mentions a forbidden classification is replaced with a structured refusal. See Chapter 6 for the conformity check.

Streaming

`POST /query/stream` is a Server-Sent Events endpoint that emits a single answer event when generation finishes. Token-by-token streaming is straightforward · wire the OpenAI streaming response to the SSE producer in `answer_generator.py` · but is not yet wired to the frontend. The same SSE machinery used for `/ingest/stream` is ready to consume it (see `backend/api/routers/query.py` and `frontend/src/utils/sse.ts`).

CHAPTER 6

Guardrails

Four guardrails sit at well-defined points in the request lifecycle. Each is independently toggleable; none is a heavyweight middleware you have to opt out of.

PII redaction

Presidio Analyzer + Anonymizer, backed by spaCy `en_core_web_sm`, run in two places:

USE_PII_REDACTION_ON_INGEST	Redact chunk content before it is embedded and stored. Anything indexed has already been sanitized; downstream retrieval cannot accidentally surface raw PII.
USE_PII_REDACTION	Redact /query input and /query output (answer + citation excerpts + retrieved chunk text) for defense in depth.

The entity list is contact and financial identifiers only: `EMAIL_ADDRESS`, `PHONE_NUMBER`, `CREDIT_CARD`, `US_SSN`, `US_BANK_NUMBER`, `IBAN_CODE`, `US_PASSPORT`, `IP_ADDRESS`, `MEDICAL_LICENSE`, `CRYPTO`. `US_DRIVER_LICENSE` and `DATE_TIME` are deliberately omitted · they produce too many false positives on job references like 'e02' and fiscal quarters like 'Q1 2024' in business documents.

THE EN_CORE_WEB_SM PIN

Presidio defaults to `en_core_web_lg`, which is ~400 MB and is downloaded on first `analyze()` call. DocuMind explicitly constructs the `AnalyzerEngine` with a `NlpEngineProvider` pointed at `en_core_web_sm` (12 MB, bundled into the Docker image). The PII entity list is regex-based; the small model is sufficient. This also silences the noisy 'Recognizer not added to registry because language is not supported' warnings and avoids a futile CUDA probe.

Taxonomy conformity

`backend/taxonomy/` runs an RDF taxonomy validator against generated answers. The taxonomy file (`data/taxonomies/`) declares preferred, allowed, and forbidden classifications; fuzzy entity linking maps answer phrases to taxonomy classes. The conformity check produces a structured `ConformityMeta` object on the response:

```
{
  "conformity": {
    "score": 0.93,
    "flagged": false,
    "forbidden_terms": []
  }
}
```

- `USE_TAXONOMY_VALIDATION=true` (default): always evaluate; expose the score and any flags in the response metadata.
- `TAXONOMY_BLOCK_FORBIDDEN=true`: when a forbidden term is detected, the answer is replaced with a structured refusal. The frontend renders a `ConformityBanner` that explains which term was the problem.
- `TAXONOMY_BLOCK_FORBIDDEN=false` (default): warn and annotate only; the answer is still returned.

Rate limiting

`slowapi` limits requests per IP. There are two ceilings, one for interactive endpoints and a much higher one for bulk uploads which may legitimately fire hundreds of requests in a minute:

`API_RATE_LIMIT_PER_MINUTE` (default 60) Applies to `POST /query`, `POST /ingest`, `POST /ingest/stream`, `GET /admin/*`.

`API_RATE_LIMIT_BULK_PER_MINUTE` (default 2000) Applies to the bulk upload endpoints and job-status polling.

Limits are computed against `X-Forwarded-For` when `nginx` is in front of the API, so real client IPs are honoured. A 429 response is returned with a `Retry-After` header; the frontend surfaces this with an exponential backoff in the bulk-upload code path.

CORS

`CORS_ALLOW_ORIGINS` is a comma-separated list. The default value `'*'` is appropriate only for local demos. In production, set explicit origins:

```
CORS_ALLOW_ORIGINS=https://docu.example.com,https://www.docu.example.com
```

When the value is anything other than `'*'`, the `CORSMiddleware` also enables `allow_credentials=True` so cookies and Authorization headers are honoured. With wildcard origins, credentials are off per the CORS spec.

CHAPTER 7

Evaluation

A RAG system that is not evaluated is mostly guesswork. DocuMind ships a small, hand-curated golden set and runs the same metrics in CI, the eval workflow, and any local checkout · so a regression is caught before it gets merged.

The golden set

tests/eval/golden_set.jsonl is a JSONL of cases over data/raw/sample_report.pdf · a synthetic but realistic annual report with text, a table, and a figure across three pages. The sample PDF is byte-reproducible via SOURCE_DATE_EPOCH.

```
{
  "id": "fig3-q4",
  "query": "What was Q4 2024 revenue according to Figure 3?",
  "doc_id": "ed7d53f9b08caa39",
  "source_pdf": "data/raw/sample_report.pdf",
  "relevant_pages": [2],
  "reference_answer": "Q4 2024 revenue was $54.6M.",
  "key_phrases": ["54.6", "Q4 2024"]
}
```

The cases deliberately span all three retrieval modalities:

Total revenue, NRR, FCF	text (page 1)
Quarterly revenue trend, DataPulse	figure caption + body (page 2)
Q3 margin, Q1 2025 row, board target	table (page 3)

If a code change improves text retrieval but breaks figure retrieval, the metrics shift visibly · that is the whole point of the modality coverage.

Metrics

Recall@5	Of the pages that should appear, how many did we return in the top 5? Averaged across the golden set.
----------	---

Hit@5	Did at least one relevant page appear in the top 5? Binary per case, averaged.
MRR	Reciprocal rank of the first relevant page. Penalises burying the right answer behind irrelevant ones.
Keyword coverage	Fraction of key_phrases that appear in the generated answer. Cheap proxy for faithfulness without an LLM judge.
p95 retrieval latency (ms)	After dropping the first cold query for warmup, p95 of the retrieve-only wall clock. Catches an $O(N^2)$ regression before it ships.
Faithfulness (Ragas)	Optional, gated by pip install -e .[eval] and RUN_RAGAS_EVAL=1. LLM judge scores whether the answer is grounded in the retrieved contexts.

CI gates

The thresholds live next to the application config so the same numbers are honoured in CI, the eval workflow, the API, and local runs:

```
# backend/core/config.py
eval_recall_at_5_threshold: float = 0.70
eval_hit_at_5_threshold:    float = 0.80
eval_mrr_threshold:        float = 0.50
eval_keyword_coverage_threshold: float = 0.66
eval_faithfulness_threshold: float = 0.85
eval_retrieval_latency_p95_ms: float = 60_000.0
```

The eval test fails (red CI) if any of these aren't met. Every threshold is also exposed as a `EVAL_*` env var so a flaky CPU runner can be tuned without rewriting the test.

Workflows

eval.yml · retrieval-eval	Weekly cron, every push touching backend/retrieval/, backend/ingestion/, tests/eval/, or backend/core/pipeline.py, and on demand. Spins up Qdrant as a service container, regenerates the sample PDF reproducibly, ingests it, runs the retrieval eval, emits a Markdown benchmark summary to the GitHub Actions run page, and uploads benchmarks.md as an artifact.
eval.yml · answer-eval	Same triggers, gated on OPENAI_API_KEY repo secret via a check-secrets job whose has_openai_key output the answer-eval job consumes via needs:. This is the documented workaround for GitHub Actions not reliably evaluating secrets.* inside if:.

Reproducing the benchmarks locally

```
( cd docker && docker compose up -d qdrant )
python scripts/generate_sample_report.py      # one-off, also tracked in git
pytest tests/eval/test_retrieval.py tests/eval/test_metrics.py -m eval -v
```

Generate the same Markdown table the workflow uploads:

```
python scripts/run_eval_report.py          # prints to stdout  
python scripts/run_eval_report.py -o bench.md # writes to file
```

CHAPTER 8

Observability

Three signals are exported by default: traces, metrics, and structured logs. Each goes to a sidecar service in the Compose stack so you do not need a SaaS account to debug a slow query.

Traces · OpenTelemetry to Jaeger

FastAPI is instrumented via `opentelemetry-instrumentation-fastapi`. Every request gets a span tree that includes:

- The HTTP server span (FastAPI).
- Retrieval spans per Qdrant collection.
- An embedding span per modality.
- A generation span around the LLM call.
- An anonymizer span around PII redaction.

Set `OTLP_ENDPOINT` to your collector. In the Compose stack this is `http://jaeger:4317`. The Jaeger UI is on `http://localhost:16686`.

Metrics · Prometheus + Grafana

FastAPI exposes `/metrics`. The Counters and Histograms in `backend/api/monitoring/metrics.py` are:

- `QUERY_REQUESTS{provider,retrieve_only}` · counter for `/query`.
- `QUERY_LATENCY` · histogram of `/query` end-to-end latency.
- `INGEST_REQUESTS` · counter for `/ingest` and `/ingest/stream`.
- `PII_REDACTIONS` · counter for every redaction that fired on input or output.

The Celery worker exposes a separate Prometheus port (`INGEST_METRICS_PORT`, default 9100) with per-job ingest counters. Both endpoints are scraped by the Prometheus container; the Grafana container ships with a dashboard preconfigured in `docker/grafana/dashboards/`.

Structured logs

structlog renders JSON to stdout. `LOG_LEVEL` controls verbosity. The common keys are `trace_id` (from OpenTelemetry), `doc_id`, `chunk_count`, and `provider`. With Docker, `docker`

compose logs -f rag-api is enough to follow a request; in production, ship to your log aggregator and key on trace_id to join across services.

Health and readiness

GET /health	Always 200 if the process is up. Useful for a TCP-level health check.
GET /ready	Probes Qdrant and Redis, and, when API_WARMUP_MODELS=true, reports whether the embedding models loaded successfully. Returns 503 if any check fails.
GET /metrics	Prometheus exposition. Not protected · bind only on the internal network in production.

CHAPTER 9

Scaling Out

Why a worker tier

Single-PDF `/ingest` runs synchronously in the API process because it is the simplest UX for the chat tab. For folders of hundreds of PDFs that is the wrong shape: a multi-hour batch should not keep a browser tab open and should not block other users from querying. DocuMind moves bulk ingest onto Celery with Redis as the broker.

The bulk-ingest protocol

1. Browser: POST `/ingest/bulk/start` with the folder name and total file count · API creates a Redis job record and returns `job_id`.
2. Browser: POST `/ingest/bulk/{job_id}/files` repeatedly, one per PDF; API saves each to `data/raw/uploads/bulk/{job_id}/`.
3. Browser: POST `/ingest/bulk/{job_id}/run` · API enqueues one Celery task per PDF onto Redis.
4. Workers: pull tasks; fingerprint each PDF; dedup against an in-process LSH cache plus the Qdrant content hash; call `RAGPipeline.ingest` on new docs.
5. Browser: GET `/ingest/bulk/{job_id}` every second; renders the progress bar.
6. Optional: POST `/ingest/bulk/{job_id}/resume` · requeue any files that errored or never started, without re-uploading.

Deduplication

Bulk ingest of a corpus that already lives in the index is expensive. Two layers of dedup prevent the obvious waste:

- Per-task: the worker computes an LSH minhash over the PDF text. If a fingerprint within `DEDUP_SIMILARITY_THRESHOLD` (default 0.85) has already been seen in the same job, the second occurrence is skipped.
- Per-corpus: each chunk's `content_hash` is checked against Qdrant before upsert. Re-ingesting an unchanged PDF is a near-no-op: parse + embed are cached in Redis (`embedding_cache_ttl_seconds = 7` days), and Qdrant rejects already-present rows.

Caching

The embedding cache is keyed by (model_name, chunk_fingerprint). On a cache hit, the embedder returns the cached vector without touching the model. This is what makes a re-ingest after a chunker change fast: only the chunks that actually changed are re-embedded.

Sizing notes

CPU only	BGE-M3 on a modern x86 server: ~50-80 chunks/sec. A 200-page PDF with text + tables produces ~600-1000 chunks; expect ~10-15 s of embedding time per doc plus parse + upsert.
GPU (RTX 3070 Ti, 8 GB)	BGE-M3 saturates the GPU at batch_size 32; ~5-8× faster than CPU. VRAM is shared between BGE-M3 (~2 GB) and CLIP (~1 GB); ColPali adds 3-4 GB and tips you over 8 GB on a -B model.
Concurrent workers	max_workers (default 4) bounds Celery prefetch. For CPU embedding, set CELERYD_CONCURRENCY to the number of physical cores.

CHAPTER 10

Deployment

Local Docker Compose

The default stack lives in `docker/docker-compose.yml`. It defines the `rag-api`, `ingest-worker`, `documind-web` (nginx), `qdrant`, `redis`, `prometheus`, `grafana`, and `jaeger` services, plus their networks and volumes.

```
cp .env.example .env
# put your OPENAI_API_KEY (or set OLLAMA_BASE_URL) in .env
cd docker
docker compose --profile production up -d --build
```

First start downloads embedding models (~15-30 minutes on a slow network, faster on subsequent rebuilds because the Hugging Face cache is a named volume). With `API_WARMUP_MODELS=true` (default), the API will not pass `/ready` until the models are loaded.

GPU build

An overlay file at `docker/docker-compose.gpu.yml` flips the build to the `cu124` PyTorch wheel and reserves NVIDIA GPUs for `rag-api` and `ingest-worker`. The host must have the NVIDIA Container Toolkit (Linux) or Docker Desktop with WSL2 GPU support (Windows).

```
cd docker
docker compose -f docker-compose.yml -f docker-compose.gpu.yml \
  --profile production up -d --build
```

TORCH VERSION PIN

`transformers 5.x` refuses to load model weights under `torch < 2.6` due to CVE-2025-32434. The Dockerfile pins `torch>=2.6.0`; the GPU overlay defaults to the `cu124` wheel index because the `cu121` index caps at `torch 2.5.1`. If your driver is too old for CUDA 12.4, set `TORCH_INDEX_URL` in your shell before building.

Fly.io

`deploy/fly.toml` ships a working Fly configuration. The defaults pick a 2-CPU/4-GB VM, mount a

volume for the Qdrant data directory, and set `API_WARMUP_MODELS=true`. The `deploy/README.md` walks through secret setup and the first deploy.

nginx in front

`frontend/nginx.conf` proxies `/api/*` to `rag-api` on the internal network and serves the static frontend bundle. Timeouts and buffering settings are tuned for two real workloads · long PDF uploads and Server-Sent Events:

<code>proxy_send_timeout / proxy_read_timeout</code>	1200 s · long PDFs on CPU can spend many minutes in parse + OCR + embed before the upstream produces a response.
<code>client_body_timeout / send_timeout</code>	1200 s · same reason on the client side.
<code>proxy_request_buffering</code>	off · stream uploads straight to <code>rag-api</code> instead of buffering the entire multipart body in nginx.
<code>proxy_buffering</code>	off · SSE (<code>/ingest/stream</code> , <code>/query/stream</code>) must flush each event immediately.
<code>client_max_body_size</code>	100 m · caps a single upload at 100 MB.

CI / CD

<code>.github/workflows/ci.yml</code>	Lints (ruff), runs pytest with Qdrant in services, type-checks the frontend, runs <code>npm ci</code> + <code>npm run build</code> . Triggers on push and pull_request for main and master.
<code>.github/workflows/eval.yml</code>	Weekly cron and path-triggered retrieval-eval. Publishes a <code>benchmarks.md</code> artifact and a GitHub Step Summary. The <code>answer-eval</code> job runs only when <code>OPENAI_API_KEY</code> is set, via the <code>check-secrets</code> guard.
<code>.github/workflows/wiki.yml</code>	Auto-syncs <code>docs/wiki/</code> · the GitHub Wiki on every push to master that touches <code>docs/wiki/**</code> .

CHAPTER 11

API Reference

Health and metrics

GET /health	Liveness probe. Always 200 if the process is up. {"status": "ok"}.
GET /ready	Readiness probe. Returns 200 only if Qdrant and Redis are reachable and (when warmup is enabled) the embedding models are loaded.
GET /metrics	Prometheus exposition. Bind on the internal network only.

Query

POST /query

Single-shot query, JSON response.

```
{
  "query": "What was Q4 2024 revenue?",
  "top_k": 5,
  "doc_id": "ed7d53f9b08caa39",
  "provider": "openai",
  "retrieve_only": false,
  "block_forbidden": null
}
```

POST /query/stream

Same body as /query, but the response is a Server-Sent Events stream. Events: status ("retrieving"), answer (the full QueryResponse payload), done.

Ingest

POST /ingest	Upload a single PDF; runs the full pipeline synchronously and returns the IngestResponse.
POST /ingest/stream	Same as /ingest but emits an SSE progress stream with stages: uploading, parsing, enriching, redacting, embedding, indexing. Terminal events: done (with the IngestResponse) or error.

POST <code>/ingest/directory</code>	Server-side directory ingest. The body specifies a path under <code>data/raw/</code> ; the API walks recursively (unless <code>recursive=false</code>) and runs the pipeline per PDF.
POST <code>/ingest/youtube/stream</code>	Ingest a YouTube lecture URL. Body: <code>{url, include_transcript, include_slides, sample_every_seconds}</code> . Same SSE stages as <code>/ingest/stream</code> ; the done payload extends <code>IngestResponse</code> with <code>video_id</code> , <code>title</code> , <code>slides_pdf_path</code> , and <code>warnings</code> . Returns 503 when <code>YOUTUBE_INGEST_ENABLED=false</code> . See Chapter 13.

Bulk ingest

POST <code>/ingest/bulk/start</code>	<code>{folder_name, total_files} · {job_id, total_files}</code> . Creates an empty bulk job in Redis.
POST <code>/ingest/bulk/{job_id}/files</code>	Upload one PDF (multipart). Returns <code>{job_id, uploaded, total, filename}</code> .
POST <code>/ingest/bulk/{job_id}/run</code>	Enqueue Celery tasks for all uploaded files in the job. Returns <code>{job_id, queued, status}</code> .
POST <code>/ingest/bulk/{job_id}/resume</code>	Re-enqueue files that errored or were never started; useful when a worker crashed mid-job.
GET <code>/ingest/bulk/{job_id}</code>	Job status: <code>total</code> , <code>uploaded</code> , <code>processed</code> , <code>ingested</code> , <code>skipped</code> , <code>failed</code> , <code>current_file</code> , <code>message</code> .

Admin

GET <code>/admin/documents</code>	List ingested documents (<code>doc_id</code> , <code>name</code> , <code>source_path</code> , <code>chunk_count</code>).
GET <code>/admin/documents/{doc_id}/suggestions</code>	Cached question suggestions for a document, used by the chat panel.
GET <code>/admin/documents/{doc_id}/file</code>	Stream the original PDF. Used by the frontend preview pane.
GET <code>/admin/directories</code>	Browse server-side directories under <code>data/raw/</code> .
GET <code>/admin/collections</code>	Qdrant collection stats: <code>name</code> , <code>points_count</code> , <code>vector_size</code> .
DELETE <code>/admin/doc/{doc_id}</code>	Remove all chunks for a document across every collection.

CHAPTER 12

Configuration Reference

Every backend setting comes from `backend/core/config.py` · a single pydantic-settings `Settings` class. Override anything via `.env` (gitignored) or real environment variables; aliases below are what you put in the env.

LLM provider

OPENAI_API_KEY	Required for OpenAI mode. Empty by default.
OPENAI_MODEL	Default gpt-4o. OpenAI chat model.
OLLAMA_BASE_URL	Default <code>http://localhost:11434</code> . In Docker: <code>http://host.docker.internal:11434</code> .
HF_TOKEN	Optional Hugging Face token for faster / gated model downloads.

Embeddings and retrieval models

TEXT_EMBEDDING_MODEL	Default BAAI/bge-m3. Multilingual text encoder.
COLPALI_MODEL	Default vidore/colqwen2-v1.0. Use vidore/colSmol-500M on CPU.
USE_COLPALI	Default false. Enable page-image retrieval; GPU recommended.
EMBEDDING_DEVICE	Default cpu. Set to cuda when a GPU is present.

Retrieval toggles

USE_HYBRID	Default false. BM25 + dense fusion on <code>text_chunks</code> . Recommended true for production.
USE_RECURSIVE_CHUNKER	Default false. Token-aware splitter.
USE_SEMANTIC_CHUNKER	Default false. Alternative to recursive · splits on cosine boundaries.
SEMANTIC_CHUNK_THRESHOLD	Default 0.75. Cosine threshold for the semantic chunker.
USE_SECTION_PATHS	Default true. Carry the heading chain on every chunk.
USE_CONTEXT_ENRICHMENT	Default true. Pre-compute neighbouring-sentence context windows at ingest.
USE_PARENT_EXPAND	Default true. On a child-chunk hit, expand to the parent passage.
USE_FLASHRANK	Default false. Lightweight cross-encoder rerank (~25 ms per 10 chunks).

Vector DB and queue

QDRANT_URL	Default http://localhost:6333.
QDRANT_API_KEY	Empty by default. Only needed for Qdrant Cloud.
REDIS_URL	Default redis://localhost:6379.
CELERY_BROKER_URL	Falls back to REDIS_URL. Override only if broker · cache.
INGEST_METRICS_PORT	Default 9100. Worker Prometheus port.
DEDUP_SIMILARITY_THRESH OLD	Default 0.85. LSH similarity above which two PDFs are treated as duplicates at bulk-ingest time.

OCR and parsing

USE_OCR	Default true. Tesseract OCR for image-only PDF pages at ingest.
OCR_LANG	Default eng. Tesseract language code.

YouTube lecture ingest

YOUTUBE_INGEST_ENABLED	Default true. When false, POST /ingest/youtube/stream returns 503.
TRANSCRIBER_PROVIDER	Default openai (Whisper; needs OPENAI_API_KEY). Set to mock for offline tests · but note the mock returns canned text for every video.
YOUTUBE_SAMPLE_EVERY_SE CONDS	Default 2.0. Frame-sampling interval for slide extraction.

API, guardrails, observability

API_RATE_LIMIT_PER_MINU TE	Default 60. Per-IP cap for /query and single-file /ingest.
API_RATE_LIMIT_BULK_PER _MINUTE	Default 2000. Per-IP cap for bulk uploads and job-status polling.
CORS_ALLOW_ORIGINS	Default *. Comma-separated origins; * disables credentials.
USE_PII_REDACTION	Default true. Presidio + spaCy redaction on /query text.
USE_PII_REDACTION_ON_IN GEST	Default true. Same redaction at ingest.
USE_TAXONOMY_VALIDATION	Default true. Run the RDF taxonomy guard on generated answers.
TAXONOMY_BLOCK_FORBIDDE N	Default false. True hard-blocks forbidden mentions; false only warns.
API_WARMUP_MODELS	Default true. Eager-load text / image / reranker models on boot.
OTLP_ENDPOINT	Default http://localhost:4317. OpenTelemetry collector.
LOG_LEVEL	Default INFO.

Evaluation thresholds

<code>EVAL_TOP_K</code>	Default 10. top_k used during the eval; metrics still cut at K=5.
<code>EVAL_RECALL_AT_5_THRESH_OLD</code>	Default 0.70.
<code>EVAL_HIT_AT_5_THRESHOLD</code>	Default 0.80.
<code>EVAL_MRR_THRESHOLD</code>	Default 0.50.
<code>EVAL_KEYWORD_COVERAGE_THRESHOLD</code>	Default 0.66.
<code>EVAL_RETRIEVAL_LATENCY_P95_MS</code>	Default 60000. Retrieve-only p95 ceiling after warmup.
<code>RUN_RAGAS_EVAL</code>	Unset by default. Set to 1 after pip install -e .[eval] for LLM-judge faithfulness.

Reproducible builds

<code>SOURCE_DATE_EPOCH</code>	Pins wall-clock leaks (PDF dates, PNG tEXt, PyMuPDF /ID). Sample report and CI default 1704067200 (2024-01-01 UTC); this book defaults to 1780272000 (2026-06-01 UTC) unless the env var is set.
--------------------------------	--

CHAPTER 13

Ingesting YouTube Lectures

PDFs are not the only thing worth asking questions about. DocuMind can ingest a YouTube lecture URL and turn it into a searchable document that behaves exactly like a PDF in the chat UI · same retrieval, same citations, same answer generation. There is no separate 'video stack': a lecture is reduced to the same DocumentChunk objects every other ingest path produces, written to the same Qdrant collections.

Two modalities from one URL

A single video yields up to two streams of chunks, which share one doc_id (youtube:{video_id} hashed) so chat retrieval spans both:

Transcript	Audio is downloaded, transcribed (OpenAI Whisper by default), and chunked with start/end timestamps. Each chunk is indexed as ChunkType.TRANSCRIPT inside the existing text_chunks collection · no new collection, no new embedder.
Slides	Video frames are sampled at a fixed interval, near-duplicate frames are dropped with perceptual hashing, the survivors are assembled into a slides.pdf, and that PDF is parsed and OCR-indexed through the very same PDF pipeline used for uploads.

Because both modalities collapse onto one doc_id, a question can be answered from the spoken narration, the on-screen slide text, or both at once · and the citation tells the user which.

The ingest path

The whole flow lives under backend/video/ and is orchestrated by YouTubeIngestor; it reuses RAGPipeline.index_chunks rather than reimplementing embedding or storage.

```

YouTube URL
-> backend/video/youtube_ingestor.py
-> transcript: download audio -> transcribe -> segments_to_chunks
               -> RAGPipeline.index_chunks
-> slides:     download video -> sample frames -> phash dedup
               -> slides.pdf -> parse -> index_chunks (clear_existing=false)
-> same Qdrant collections, embeddings, retrieval, and generation as PDFs

```

The slide branch passes clear_existing=false so it adds to, rather than replaces, the transcript

chunks already indexed under the same `doc_id`. Re-ingesting the same URL overwrites the previous vectors because the `doc_id` is derived from the video id.

Slide-scoped questions

The reason to extract slides at all is to answer a very specific kind of question: 'What did the author say about slide 7?'. When slide extraction ran, DocuMind detects the slide N / page N reference and, instead of a generic semantic search, returns a focused pair:

1. Slide N's on-screen text · the OCR'd content of that slide.
2. The transcript spoken while slide N was visible · the segments bounded by [slide N timestamp, slide N+1 timestamp).

This works because slide chunks store `metadata.timestamp` (when the slide appeared) and transcript chunks store `start_time` / `end_time`. The retriever joins them on time, tags the spoken chunks with `aligned_slide_number`, and the answer generator labels them 'Spoken during slide N' so the LLM summarizes the narration for that slide rather than returning an unrelated semantic hit.

- Slide-scoped queries return only that slide plus its narration · no unrelated semantic results · which keeps citations tight.
- If no narration was indexed for a slide's time window (a silent title card, say), the answer describes the on-screen text and notes that narration was not indexed for that segment.
- Scoping applies to video documents only; for PDFs, 'page N' stays an ordinary semantic query.

Citations and preview

YouTube-specific fields ride along in `chunk.metadata` and surface in the API citations, so the frontend can render meaningful badges and deep links:

Transcript citation	<code>start_time</code> , <code>end_time</code> , <code>youtube_url</code> · rendered like 'Transcript 8:12-8:55' and linked to the watch URL at the right timestamp.
Slide citation	<code>slide_number</code> , <code>timestamp</code> , <code>youtube_url</code> · rendered like 'Slide 6, extracted around 8:20'. A slide hit with a generated PDF also exposes <code>page_number</code> for an in-app preview jump.

A YouTube document stores `source_path` as the virtual path `youtube:{video_id}`, not a file on disk. When slides were extracted, the in-app preview resolves to `data/processed/youtube/{video_id}/slides.pdf`. A transcript-only lecture has no PDF, so the preview shows an explanatory message instead of an error · chat still works from the indexed transcript.

Operating it

The endpoint mirrors the PDF SSE ingest: `POST /ingest/youtube/stream` accepts the URL and the two modality toggles, then streams progress (`progress` / `done` / `error`) just like `/ingest/stream`. The done payload extends the normal `IngestResponse` with `video_id`, `title`, `slides_pdf_path`, and optional warnings.

```
curl -N -X POST http://localhost:8002/ingest/youtube/stream \
-H "Content-Type: application/json" \
-H "Authorization: Bearer <session-token>" \
-d '{
  "url": "https://www.youtube.com/watch?v=VIDEO_ID",
  "include_transcript": true,
  "include_slides": true,
  "sample_every_seconds": 2.0
}'
```

Three settings govern the feature (full table in Chapter 12): `YOUTUBE_INGEST_ENABLED` gates the endpoint (503 when false), `TRANSCRIBER_PROVIDER` selects openai Whisper or a mock for tests, and `YOUTUBE_SAMPLE_EVERY_SECONDS` sets the frame-sampling interval. The Docker image additionally needs `ffmpeg`, `yt-dlp`, `opencv-python-headless`, and `imagehash`, all installed in `docker/Dockerfile`.

THE MOCK TRANSCRIBER PITFALL

`TRANSCRIBER_PROVIDER=mock` returns the same two hardcoded segments (~28 s of placeholder text) for every video, regardless of audio · handy for Docker smoke tests, misleading in real use. If a lecture shows only one or two transcript chunks of unrelated content, it was almost certainly ingested under mock. Set `TRANSCRIBER_PROVIDER=openai`, recreate the API container, and re-ingest the URL; the doc keeps its `doc_id`, so the re-ingest overwrites the mock vectors.

Limitations

- Only ingest videos you have the right to process. YouTube support targets personal knowledge management, authorized lectures, and content you own or are permitted to analyze.
- Slide deduplication uses perceptual hashing; fast scene changes or animated slides can produce extra pages.
- Transcription quality and cost depend on the configured provider (openai Whisper by default).
- Very long videos increase download, transcription, and indexing time; CPU-only hosts should expect multi-minute ingests.

APPENDIX

Appendix A · Repository Layout

A bird's-eye view of every top-level directory in the repository.

```
RAG/
|-- backend/
|   |-- api/                # FastAPI routers, schemas, monitoring, guardrails
|   |   |-- routers/        # health, query, ingest, bulk_ingest, youtube, admin
|   |   n
|   |   |-- monitoring/     # metrics.py, tracing.py
|   |   |-- guardrails/     # pii.py (Presidio + spaCy)
|   |   |-- main.py         # FastAPI app + lifespan + CORS + rate limiting
|   |   \-- rate_limit.py
|   |-- core/               # pipeline + config + shared models
|   |   |-- pipeline.py     # RAGPipeline -- the single entry point
|   |   |-- config.py       # pydantic-settings Settings
|   |   \-- models.py       # DocumentChunk, RetrievedContext, QueryResponse, ...
|   ..
|   |-- ingestion/
|   |   |-- parsers/        # PDFTextParser, TableParser, FigureParser, ImageParser
|   |   |   |-- embeddings/ # TextEmbedder, ImageEmbedder, ColPaliEmbedder
|   |   |   \-- stores/     # qdrant_store.py
|   |   |-- retrieval/
|   |   |   |-- multimodal_retriever.py
|   |   |   |-- hybrid_retriever.py
|   |   |   |-- parent_expand.py
|   |   |   |-- asset_refs.py
|   |   |   |-- cross_encoder_reranker.py
|   |   |   |-- flashrank_reranker.py
|   |   |   |-- chunking/    # recursive + semantic chunkers
|   |   |   |-- preprocessing.py # ingest-time enrichment dispatcher
|   |   |   \-- chunk_filters.py
|   |   |-- generation/
|   |   |   \-- answer_generator.py
|   |   |-- video/          # YouTube ingest: download, transcribe, slides
|   |   |   |-- youtube_ingestor.py # orchestrates transcript + slide indexing
|   |   |   |-- transcriber.py     # whisper / mock providers
|   |   |   |-- frame_sampler.py    # sample + phash-dedup video frames
|   |   |   \-- slide_pdf_builder.py # assemble sampled frames into slides.pdf
|   |   |-- scaling/
|   |   |   |-- workers.py        # Celery tasks (ingest_single_pdf_task, ...)
|   |   |   \-- pipeline/         # ingest modes, scalable_ingest
```

```

| | \-- embedding/          # cached_embed.py
| \-- taxonomy/            # RDF taxonomy validation, conformity
|-- frontend/
| | \-- src/
| | | \-- api/client.ts    # typed RagApiClient
| | | \-- components/     # PdfUploader, ChatPanel, CitationsPanel, ...
| | | \-- utils/          # sse.ts, renderLatex.ts, apiErrors.ts
| | \-- App.tsx
| \-- nginx.conf
|-- docker/
| | \-- Dockerfile         # rag-api + ingest-worker image
| | \-- docker-compose.yml # base stack
| | \-- docker-compose.gpu.yml # GPU overlay (cu124)
| \-- grafana/dashboards/  # preconfigured Grafana dashboard
|-- docs/
| | \-- wiki/              # version-controlled wiki source
| \-- documind-book.pdf    # this book
|-- tests/
| | \-- eval/              # golden set + metrics + retrieval eval
| | \-- api/               # router-level tests
| | \-- ingestion/, retrieval/, scaling/, taxonomy/
| \-- test_pipeline.py     # RAGPipeline unit tests
|-- scripts/
| | \-- generate_sample_report.py # byte-reproducible sample PDF
| | \-- run_eval_report.py      # Markdown benchmark publisher
| | \-- generate_bulk_pdfs.py   # corpus generator for load tests
| \-- generate_book.py         # this script
|-- deploy/
| \-- fly.toml               # working Fly.io deployment
|-- .github/workflows/      # ci.yml, eval.yml, wiki.yml
|-- data/
| | \-- raw/                # input PDFs (sample_report.pdf is tracked)
| | \-- processed/          # rendered pages and figures
| \-- taxonomies/           # RDF/OWL taxonomy files
|-- README.md
|-- pyproject.toml
\-- .env.example

```

APPENDIX

Appendix B · Troubleshooting

/query returns 503 with a torch upgrade message

The embedding stack failed to load because torch is below 2.6 while transformers is 5.x. Rebuild the API container:

```
cd docker
docker compose -f docker-compose.yml -f docker-compose.gpu.yml \
  --profile production build rag-api ingest-worker
docker compose -f docker-compose.yml -f docker-compose.gpu.yml \
  --profile production up -d rag-api ingest-worker
```

/query returns 'Internal Server Error'

Check docker logs rag_api for a stack trace. The two common causes:

- torch/transformers mismatch · see the previous section.
- Qdrant or Redis is unreachable · GET /ready will tell you which check is failing.

PDFParseError on a specific page

The page produced no parseable text and no images. Causes: a scanned-only PDF without OCR (set USE_OCR=true and rebuild), or an intentionally blank page. The pipeline logs the page number and the doc_id; you can either re-OCR or accept the gap.

Long PDF upload returns 504 with raw HTML in the UI

Pre-0.1.0 builds did not parse nginx HTML error pages and dumped the raw body into the UI. Update to the current frontend (parseApiErrorBody in frontend/src/utils/apiErrors.ts) and rebuild the documind-web container. The frontend now shows a single sentence and points the user at the async bulk-upload path or a GPU build.

Bulk ingest hangs on a single file

Look at the worker log for the `doc_id` in question. Common culprits: an encrypted PDF (presented to the user as a parse error), a scanned-only PDF without OCR, or a single >100 MB page that pushes embedding past available VRAM. The bulk endpoint exposes resume semantics; you can skip the offending file and continue.

Sample PDF differs across machines

`SOURCE_DATE_EPOCH` is the only knob that should change the output. If the sample PDF generated by `scripts/generate_sample_report.py` differs between machines despite the env var, check `matplotlib` + `PyMuPDF` versions; the pinned set in `pyproject.toml` is the only validated combination.

APPENDIX

Appendix C · Glossary

BGE-M3	BAAI's multilingual text embedding model. Default text encoder. 1024-dim, supports long context.
CLIP	Contrastive image-text encoder. Used to embed figure and table images into a shared text/image space.
ColPali	Visual document encoder (vidore/colqwen2 or vidore/colSmol). Embeds whole rendered PDF pages; opt-in via USE_COLPALI=true.
Chunk	An atomic unit of retrievable content: a text passage, a table, a figure, or a page image.
Citation	A pointer from an answer to a specific chunk (doc_id, page_number, chunk_id) so the user can verify the claim.
Cross-encoder	A model that scores a (query, document) pair jointly. Used as an optional reranker. More accurate than bi-encoders, much slower.
Dense retrieval	Vector-similarity search over learned embeddings. Strong on semantics and paraphrase.
Eval gate	A CI threshold above which a metric must remain. A regression below the gate fails CI.
FlashRank	A lightweight cross-encoder reranker (~25 ms per 10 chunks). Good quality-per-latency ratio.
Golden set	The hand-curated JSONL of evaluation cases. Each case has a query, the relevant pages, and key phrases to find in the answer.
Hit@K	Binary metric: did at least one relevant page appear in the top K? Averaged across cases.
Hybrid retrieval	Combination of dense and sparse (BM25) retrieval, fused with RRF.
LSH	Locality-sensitive hashing. Used in bulk-ingest dedup to detect near-duplicate PDFs cheaply.
MRR	Mean Reciprocal Rank. $1 / \text{rank}$ of the first relevant page, averaged across cases.
OCR	Optical character recognition (Tesseract). Used at ingest for image-only PDF pages.
OTLP	OpenTelemetry Protocol. The wire format used to ship traces from the API to Jaeger.
Parent expansion	On a small-chunk hit, expand to the neighbouring sibling passages to give the LLM more context without losing retrieval precision.

Presidio	Microsoft's PII analyzer and anonymizer. Wraps spaCy and a set of regex / pattern recognizers.
Qdrant	Open-source vector database. DocuMind uses four collections · text_chunks, table_chunks, figure_chunks, page_chunks.
RAG	Retrieval-Augmented Generation. The pattern this whole repository implements.
RRF	Reciprocal Rank Fusion. A rank-based score combiner: $1 / (k + \text{rank})$, summed across retrievers. k=60 by default. Robust because it ignores raw score scales.
Slide alignment	Joining transcript chunks to slides by time: the narration spoken during [slide N timestamp, slide N+1 timestamp) is tagged aligned_slide_number=N and labeled 'Spoken during slide N' for the LLM. Powers slide-scoped questions (Chapter 13).
SSE	Server-Sent Events. The transport DocuMind uses for /ingest/stream progress and (in the future) /query/stream token streaming.
Transcript chunk	A timestamped slice of a transcribed video's audio (ChunkType.TRANSCRIPT), stored in the text_chunks collection under a youtube:{video_id} doc_id. Carries start_time / end_time for time-based slide alignment.
Stable doc ID	The deterministic 16-character SHA-256 prefix of the path-from-raw of a PDF. The primary key for everything related to a document.
Section path	The heading chain ('Executive Summary > Q4 Highlights') attached to each chunk by USE_SECTION_PATHS=true. Improves both retrieval and the LLM's situational awareness.
Taxonomy guard	The RDF-based conformity check that scores generated answers against allowed and forbidden classifications.